HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1
_____ o0o _____

BÀI TẬP NHÓM TRUY XUẤT THÔNG TIN

Tên đề tài: Truy xuất thông tin từ văn bản luật Việt Nam

Nhóm lớp : 02
Số thứ tự nhóm: 21

Vũ Công Tuấn Dương MSV: B22DCKH024

HÀ NỘI, 04/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Mục tiêu và phạm vi đề tài 1

1.2 Bố cục báo cáo 1

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 2

2.1 Tiền xử lý văn bản 2

2.1.1 Tách từ (Tokenization) 2

2.1.2 Chuẩn hóa văn bản (Text Normalization) 3

2.1.3 Làm sạch văn bản (Text Cleaning) 4

2.1.4 Loại bỏ từ dừng (Stopword Removal) 4

2.1.5 Chỉnh sửa từ (Text Correction) 5

2.2 Chỉ mục (Indexing) 5

2.2.1 Chỉ mục ngược (Inverted Index) 6

2.2.2 Các bước lập chỉ mục ngược 7

2.2.3 Chỉ mục vị trí (Positional Index) 7

2.3 Các mô hình chấm điểm mức độ liên quan của tài liệu 8

2.3.1 Mô hình Boolean 8

2.3.2 TF-IDF 9

2.3.3 BM25 10

2.4 Các thước đo đánh giá 12

2.4.1 Precision@K (Precision at K) 12

2.4.2 Recall at K (Recall at K) 12

2.4.3 F1 at K (F1 at K) 13

2.4.4 Mean Reciprocal Rank (MRR) 13

2.4.5 Mean Average Precision (MAP) 14

2.5 Elasticsearch 15
2.5.1 Các thành phần chính 16
2.5.2 Cách Elasticsearch đánh chỉ mục 16
2.5.3 Cách Elasticsearch truy vấn..... 17
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 20
3.1 Tập dữ liệu 20
3.1.1 Tập văn bản 20
3.1.2 Tập dữ liệu huấn luyện và tập dữ liệu kiểm thử..... 21
3.2 Xây dựng chỉ mục Elasticsearch 22
3.3 Xây dựng chỉ mục ngược 25
3.3.1 Quy trình xây dựng chỉ mục ngược 25
3.3.2 Tối ưu hóa chỉ mục 25
3.3.3 Các phép toán trên chỉ mục ngược 25
3.4 Các phương pháp truy xuất 27
3.4.1 Mô hình Boolean 27
3.4.2 Mô hình không gian vector (VSM) 30
3.4.3 Mô hình BM25 30
3.5 Kết quả và đánh giá 32
3.5.1 Kết quả đánh giá tổng quan 33
3.5.2 Nhận xét kết quả 34
3.6 Giao diện người dùng (UI)..... 36
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 39
4.1 Tổng kết..... 39
4.2 Kết quả đạt được..... 39
4.3 Hạn chế 39
4.4 Hướng phát triển..... 40

TÀI LIỆU THAM KHẢO..... 41

DANH MỤC HÌNH VẼ

Hình 2.1 Ví dụ về chỉ mục ngược và các thành phần của nó 6
Hình 2.2 Các bước lập chỉ mục trong Elasticsearch 17
Hình 2.3 Các bước truy vấn trong Elasticsearch 19

Hình 3.1 Giao diện chương trình minh họa kết quả 38

DANH MỤC BẢNG BIỂU

Bảng 3.1 So sánh trường `content` và trường `content.keyword` . . 24
Bảng 3.2 Kết quả đánh giá tổng quan các phương pháp 33
Bảng 3.3 Kết quả đánh giá mô hình BM25 33
Bảng 3.4 Kết quả đánh giá mô hình Boolean 34
Bảng 3.5 Kết quả đánh giá mô hình VSM 34
Bảng 3.6 Kết quả đánh giá ES với chỉ mục đã được xử lý 34
Bảng 3.7 Kết quả đánh giá ES với chỉ mục bình thường 34

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Mục tiêu và phạm vi đề tài

Với phạm vi và mục tiêu của môn học - trích xuất thông tin tập trung vào những phương pháp để trích xuất được thông tin từ dữ liệu, đề tài này sẽ tập trung vào việc cài đặt những phương pháp cơ bản khác nhau và so sánh với nhau cũng như sử dụng một công nghệ làm đường cơ sở để tham chiếu, so sánh với những phương pháp để cài đặt. Trong đề tài này sẽ sử dụng Elasticsearch - một công cụ tìm kiếm phân tán làm đường cơ sở để so sánh với những phương pháp được cài đặt - mô hình boolean, mô hình không gian vector sử dụng TF-IDF, mô hình sử dụng BM25 để xếp hạng mức độ liên quan của các tập tài liệu

1.2 Bố cục báo cáo

Chương 2 sẽ trình bày về cơ sở lý thuyết liên quan và chương 3 trình bày về kết quả đánh giá cũng như giao diện để trình bày kết quả đạt được. Chương 4 sẽ trình bày về kết luận kết quả đạt được và đưa ra hướng phát triển cho đề tài

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tiền xử lý văn bản

Tiền xử lý văn bản (Text Preprocessing) là bước quan trọng và không thể thiếu trong bất kỳ hệ thống xử lý ngôn ngữ tự nhiên và truy xuất thông tin. Mục đích chính của bước này là loại bỏ nhiễu (noise), chuẩn hóa dữ liệu đầu vào, và chuyển đổi văn bản thô thành định dạng phù hợp cho các thuật toán xử lý tiếp theo. Một văn bản thường chứa nhiều yếu tố không cần thiết như ký tự đặc biệt, số, URL, HTML tags, và các từ không mang ý nghĩa ngữ nghĩa. Việc loại bỏ các yếu tố này giúp cải thiện đáng kể chất lượng của các mô hình truy xuất thông tin [1].

2.1.1 Tách từ (Tokenization)

Tách từ (Tokenization) là quá trình chia một chuỗi văn bản thành các đơn vị nhỏ hơn có ý nghĩa, gọi là "token". Token thường là từ, nhưng cũng có thể là số, dấu câu, hoặc các đơn vị có ý nghĩa khác tùy thuộc vào mục đích sử dụng. Bước tách từ là bước đầu tiên và cơ bản nhất trong hầu hết các tác vụ xử lý ngôn ngữ tự nhiên. Nếu tách từ không chính xác, các bước xử lý sau sẽ bị ảnh hưởng nghiêm trọng, dẫn đến kết quả không chính xác.

Có nhiều phương pháp tách từ khác nhau, mỗi phương pháp có ưu điểm và nhược điểm riêng:

- **Tách theo khoảng trắng (Whitespace Tokenization):** Phương pháp đơn giản nhất, chia chuỗi theo dấu cách. Đây là cách đơn giản, dễ thực hiện nhưng có nhược điểm khi không xử lý được nhiều trường hợp phức tạp như: dấu câu dính liền từ ("xin chao,"), từ ghép ("cong nghe thong tin"), và các trường hợp đặc biệt khác.
- **Tách theo dấu câu (Punctuation-based Tokenization):** Chia chuỗi theo các dấu câu như dấu chấm, phẩy, chấm hỏi, v.v. Phương pháp này tốt hơn tách theo khoảng trắng nhưng vẫn gặp vấn đề với các dấu câu nằm trong từ hoặc viết tắt.
- **Tách dựa trên từ điển (Dictionary-based Tokenization):** Sử dụng một từ điển các từ đã biết để nhận diện và tách các từ. Phương pháp này đơn giản, dễ cài đặt nhưng vẫn có các nhược điểm như phụ thuộc vào từ điển, độ chính xác không cao đối với từ mới, và không giải quyết được trường hợp từ không có trong từ điển.
- **Tách dựa trên quy tắc (Rule-based Tokenization):** Áp dụng các quy tắc ngữ pháp hoặc hình thái học để tách từ (ví dụ: tách tiền tố, hậu tố trong tiếng Anh

như "un-", "-ing"). Phương pháp này đòi hỏi hiểu biết sâu về ngữ pháp của ngôn ngữ.

- **Tách dựa trên học máy (Machine Learning Tokenization):** Sử dụng các mô hình học máy (như CRF, HMM, Neural Network) để dự đoán ranh giới từ. Đây là phương pháp tiên tiến nhất, cho độ chính xác cao nhất nhưng đòi hỏi dữ liệu huấn luyện và chi phí tính toán lớn.

Đối với tiếng Việt, có một số khó khăn đặc thù khi tách từ do đặc điểm của ngôn ngữ:

- **Từ ghép (Compound words):** Tiếng Việt có nhiều từ ghép như "công nghệ thông tin", "bộ máy tính toán". Việc xác định ranh giới từ trong từ ghép rất khó vì không có khoảng trắng phân cách.
- **Từ viết tắt (Abbreviations):** Nhiều từ viết tắt trong văn bản pháp luật như "TV", "BH", "TP.HCM" gây khó khăn cho việc tách từ.
- **Dấu câu dính liền với từ:** Trong tiếng Việt, dấu câu thường được đặt dính liền với từ đứng trước (ví dụ: "xin chào,") gây khó khăn cho việc xử lý.
- **Số đi kèm đơn vị đo:** Các trường hợp như "10kg", "50km/h", "100.000 đồng" cần được xử lý đặc biệt để giữ nguyên ý nghĩa.

Các công cụ hỗ trợ tách từ tiếng Việt thường sử dụng các phương pháp học máy như CRF (Conditional Random Fields) hoặc Deep Learning để nhận diện ranh giới từ với độ chính xác cao.

2.1.2 Chuẩn hóa văn bản (Text Normalization)

Chuẩn hóa văn bản là quá trình chuyển đổi văn bản về một dạng thống nhất để đảm bảo tính nhất quán. Mục đích là loại bỏ các biến thể của cùng một từ để các thuật toán có thể nhận ra chúng như cùng một khái niệm. Các bước chuẩn hóa thường bao gồm:

- **Chuyển đổi chữ hoa/chữ thường:** Chuyển tất cả các ký tự về cùng một dạng (thường là chữ thường) để đảm bảo "ViệtNam" và "việtnam" được nhận ra như cùng một từ. Tuy nhiên, trong một số trường hợp riêng danh cần được giữ nguyên (ví dụ: tên riêng).
- **Xử lý dấu gạch nối:** Quyết định có loại bỏ dấu gạch nối hay giữ lại. Ví dụ: "truy-xuất" có thể được chuyển thành "truy xuất".
- **Xử lý Unicode:** Chuẩn hóa các ký tự Unicode về cùng một dạng (ví dụ: chuyển "à" từ mã tổ hợp sang mã dựng sẵn).
- **Xử lý từ viết tắt:** Mở rộng các từ viết tắt về dạng đầy đủ nếu cần thiết.

Các công cụ chuẩn hóa văn bản tiếng Việt thường cung cấp các hàm tự động để thực hiện các bước chuẩn hóa này.

2.1.3 Làm sạch văn bản (Text Cleaning)

Làm sạch văn bản là bước loại bỏ các yếu tố không mong muốn hoặc không mang ý nghĩa ngữ nghĩa từ văn bản. Mục đích là giảm nhiễu và tập trung vào nội dung chính. Một số loại nhiễu thường gặp trong văn bản:

- **HTML tags và XML tags:** Các thẻ HTML như `<p>`, `<div>`, `<xml>` thường xuất hiện trong văn bản được crawl từ web. Các thẻ này cần được loại bỏ hoàn toàn.
- **Đường dẫn URL:** Các liên kết web như "https://example.com" không mang ý nghĩa ngữ nghĩa trong hầu hết các bài toán NLP.
- **Ký tự đặc biệt và biểu tượng cảm xúc (emojis):** Các ký tự đặc biệt như #, @, *, và các biểu tượng cảm xúc không mang ý nghĩa cho việc truy xuất thông tin pháp luật.
- **Số:** Trong một số trường hợp cụ thể (như phân tích cảm xúc), số có thể không cần thiết. Tuy nhiên, trong văn bản pháp luật, số thường mang ý nghĩa quan trọng (điều khoản, mức phạt, v.v.) nên cần cân nhắc kỹ.
- **Khoảng trắng thừa:** Nhiều khoảng trắng thừa giữa các từ gây ảnh hưởng đến kết quả tách từ.
- **Ký tự không phải ASCII:** Trong tiếng Việt, cần giữ lại các ký tự có dấu (à, ả, ạ, ...). Tuy nhiên, các ký tự đặc biệt khác cần được loại bỏ.

Tùy vào bài toán cụ thể mà có thể linh hoạt xác định và loại bỏ các yếu tố nhiễu phù hợp.

2.1.4 Loại bỏ từ dừng (Stopword Removal)

Từ dừng (stopword) là các từ rất phổ biến trong ngôn ngữ nhưng thường không mang nhiều ý nghĩa phân biệt hoặc ngữ nghĩa quan trọng cho các tác vụ xử lý ngôn ngữ tự nhiên. Trong tiếng Việt, các từ dừng phổ biến bao gồm: "là", "và", "của", "trong", "với", "được", "có", "các", ... Việc loại bỏ các từ dừng mang lại nhiều lợi ích:

- **Giảm kích thước dữ liệu:** Loại bỏ từ dừng giúp giảm đáng kể kích thước của tập dữ liệu, đặc biệt quan trọng khi làm việc với tập dữ liệu lớn.
- **Giảm thời gian xử lý:** Thời gian xây dựng chỉ mục và truy vấn được giảm đáng kể khi không phải xử lý các từ dừng.

- **Cải thiện hiệu suất mô hình:** Nhiều nghiên cứu cho thấy việc loại bỏ từ dừng giúp cải thiện chất lượng của các mô hình truy xuất thông tin.
- **Giảm nhiễu:** Các từ dừng xuất hiện rất thường xuyên trong hầu hết các tài liệu, gây nhiễu cho việc tính điểm liên quan.

Việc xây dựng từ điển từ dừng cho từng ngôn ngữ là cần thiết để đảm bảo hiệu quả của bước lọc này.

Về các ký tự đặc biệt, việc loại bỏ những ký tự này cũng để giảm nhiễu. Tuy nhiên, cần cân nhắc ngữ cảnh. Ví dụ: trong phân tích cảm xúc, dấu chấm than "!" có thể mang ý nghĩa tích cực mạnh. Trong văn bản pháp luật, dấu chấm phẩy ";" thường ngăn cách các điều khoản nên mang ý nghĩa quan trọng.

2.1.5 Chỉnh sửa từ (Text Correction)

Trong văn bản có thể có một số lỗi chính tả, lỗi gõ phím hoặc các biến thể không chuẩn của từ. Do đó cần phải sửa để đưa về dạng chuẩn để đảm bảo tính nhất quán của dữ liệu. Lỗi chính tả có thể tạo ra nhiều "từ" khác nhau cho cùng một khái niệm, làm giảm hiệu quả của các mô hình truy xuất. Có một số phương pháp để chỉnh sửa từ:

- **Dựa trên từ điển (Dictionary-based Correction):** So sánh từ với một từ điển chuẩn, tìm từ gần nhất bằng cách sử dụng các thuật toán khoảng cách Levenshtein, Jaro-Winkler, v.v. Phương pháp này đơn giản nhưng phụ thuộc vào từ điển.
- **Dựa trên mô hình ngôn ngữ (Language Model Correction):** Sử dụng các mô hình thống kê (n-gram) hoặc học sâu (Neural Network) để dự đoán từ đúng dựa trên ngữ cảnh xung quanh. Phương pháp này cho độ chính xác cao hơn nhưng đòi hỏi dữ liệu huấn luyện.
- **Áp dụng quy tắc (Rule-based Correction):** Áp dụng các quy tắc để sửa các lỗi phổ biến. Ví dụ: sửa "nghiep" thành "nghiệp" dựa trên quy tắc ghép dấu.

Các phương pháp chỉnh sửa từ sử dụng các quy tắc ngữ pháp của từng ngôn ngữ để chuyển đổi các từ về dạng chuẩn.

2.2 Chỉ mục (Indexing)

Sau khi tiến hành tiền xử lý để tách từ, bước tiếp theo là xây dựng chỉ mục từ từ điển chứa các từ đã được tách. Chỉ mục (Index) là cấu trúc dữ liệu cho phép truy xuất nhanh chóng các tài liệu chứa một từ hoặc cụm từ cụ thể. Trong hệ thống truy xuất thông tin, chỉ mục đóng vai trò then chốt quyết định tốc độ và hiệu quả của việc tìm kiếm.

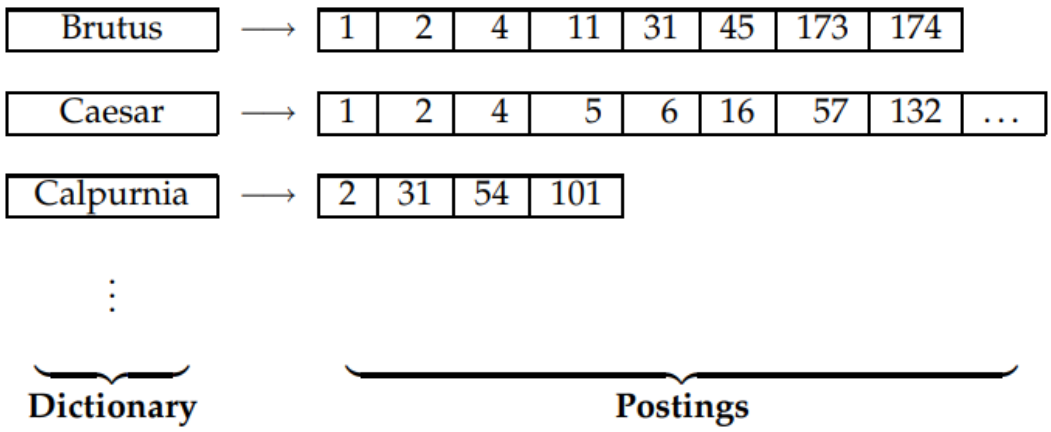
2.2.1 Chỉ mục ngược (Inverted Index)

Chỉ mục ngược (Inverted Index) là cấu trúc chỉ mục phổ biến nhất trong các hệ thống truy xuất thông tin. Khác với chỉ mục thuận (Forward Index) lưu trữ tài liệu theo thứ tự, chỉ mục ngược ánh xạ ngược từ từ (term) đến tài liệu mà từ đó xuất hiện [1]. Ý tưởng chính của chỉ mục ngược là giữ một tập từ điển các từ (thường được gọi là tập từ vựng - vocabulary). Với mỗi từ, xây dựng một danh sách chứa các tài liệu từ đó xuất hiện. Mỗi phần tử trong danh sách còn được gọi là posting. Danh sách chứa các posting gọi là postings list và tập hợp các postings list được gọi là postings.

Cấu trúc của chỉ mục ngược bao gồm hai thành phần chính:

1. **Từ điển (Dictionary):** Tập hợp tất cả các từ khác nhau xuất hiện trong tập tài liệu. Mỗi từ trong từ điển thường đi kèm với các thống kê như document frequency (số lượng tài liệu chứa từ đó).
2. **Postings:** Danh sách các tài liệu chứa từ đó, thường được sắp xếp theo ID tài liệu và có thể bao gồm thông tin bổ sung như tần suất xuất hiện, vị trí trong tài liệu.

Hình 2.1 minh họa một ví dụ về chỉ mục ngược và các thành phần của nó.



Hình 2.1: Ví dụ về chỉ mục ngược và các thành phần của nó

Ưu điểm của chỉ mục ngược:

- **Tìm kiếm nhanh:** Truy vấn một từ có thể được thực hiện trong O(1) hoặc O(log M) với M là kích thước từ điển, thay vì quét toàn bộ tài liệu O(N).
- **Xử lý truy vấn Boolean dễ dàng:** Các phép toán giao (AND), hợp (OR), bù (NOT) có thể được thực hiện trên các postings lists.

- **Hỗ trợ xếp hạng:** Thông tin tần suất trong postings cho phép tính điểm liên quan.
- **Mở rộng tốt:** Có thể dễ dàng thêm tài liệu mới mà không cần xây dựng lại toàn bộ chỉ mục.

Nhược điểm:

- **Tiêu tốn bộ nhớ:** Cần lưu trữ cả từ điển và postings.
- **Chi phí xây dựng ban đầu:** Xây dựng chỉ mục ngược tốn thời gian.
- **Cập nhật phức tạp:** Việc cập nhật chỉ mục (thêm/sửa/xóa tài liệu) phức tạp hơn so với chỉ mục thuận.

2.2.2 Các bước lập chỉ mục ngược

Quá trình xây dựng chỉ mục ngược bao gồm các bước cơ bản sau [1]:

1. **Tạo chuỗi các cặp (token, ID tài liệu):** Từ văn bản gốc, sau khi tiền xử lý, tạo ra danh sách các cặp (token đã chỉnh sửa, ID tài liệu).
2. **Sắp xếp theo thuật ngữ và ID tài liệu:** Sắp xếp danh sách theo thuật ngữ, sau đó theo ID tài liệu. Đây là bước cốt lõi của việc lập chỉ mục, quyết định cấu trúc cuối cùng của chỉ mục.
3. **Gộp các mục trùng lặp:** Gộp các mục trùng lặp của một từ (thuật ngữ) trong cùng một tài liệu, tính tổng tần suất.
4. **Tách thành từ điển và postings:** Chỉ mục được tách thành hai phần: từ điển (các từ duy nhất) và postings lists (danh sách tài liệu).
5. **Bổ sung thông tin tần suất:** Lưu trữ thêm thông tin tần suất của từ trong mỗi tài liệu và tổng số tài liệu chứa từ (document frequency) để phục vụ cho việc xây dựng mô hình không gian vector và BM25.

Thuật toán xây dựng chỉ mục ngược có độ phức tạp $O(N \times L \times log(N \times L))$ với N là số tài liệu và L là độ dài trung bình của tài liệu.

2.2.3 Chỉ mục vị trí (Positional Index)

Chỉ mục vị trí (Positional Index) được xây dựng như một bản nâng cấp của chỉ mục ngược. Cấu trúc tương tự như chỉ mục ngược, nhưng khác biệt ở chỗ trong danh sách postings, lưu trữ cả vị trí xuất hiện của mỗi thuật ngữ trong tài liệu. Cấu trúc của chỉ mục vị trí:

$$\text{Từ}_k : [(doc_1, [pos_1, pos_2, ..., pos_m]), (doc_2, [pos_1, pos_2, ..., pos_n]), ...]$$

Ví dụ về một chỉ mục vị trí: angels: [doc2 -> [36, 174, 252, 651], doc4 -> [12, 22, 102, 432], doc7 -> [17]]

Trong đó, "doc2" là ID của tài liệu và danh sách bên trong chứa thứ tự vị trí của từ "angels" xuất hiện trong văn bản.

Ứng dụng của chỉ mục vị trí:

- **Truy vấn cụm từ (Phrase Query):** Cho phép tìm các tài liệu chứa một cụm từ theo đúng thứ tự. Ví dụ: truy vấn "tin học" chỉ trả về các tài liệu có "tin" đứng ngay trước "học".
- **Truy vấn gần (Proximity Query):** Cho phép tìm các tài liệu có hai từ cách nhau trong một khoảng cách nhất định.
- **Tính điểm nâng cao:** Có thể ưu tiên các từ xuất hiện gần nhau trong tài liệu.

Tuy nhiên, chỉ mục vị trí tiêu tốn nhiều bộ nhớ hơn chỉ mục ngược không có vị trí, đặc biệt với các tài liệu dài.

2.3 Các mô hình chấm điểm mức độ liên quan của tài liệu

Trong hệ thống truy xuất thông tin, sau khi tìm được các tài liệu chứa các từ trong truy vấn, cần một phương pháp để xếp hạng (ranking) các tài liệu đó theo mức độ liên quan với truy vấn. Các mô hình chấm điểm (scoring models) được sử dụng để tính điểm liên quan cho mỗi tài liệu.

2.3.1 Mô hình Boolean

Mô hình Boolean là mô hình truy xuất cơ bản nhất, sử dụng các toán tử logic (AND, OR, NOT) để kết hợp các điều kiện tìm kiếm. Tài liệu được coi là liên quan hoặc không liên quan (binary), không có xếp hạng.

Ưu điểm:

- **Đơn giản:** Dễ hiểu và dễ cài đặt.
- **Chính xác:** Khi người dùng viết đúng truy vấn, kết quả chính xác.
- **Hiệu quả cho tìm kiếm chính xác:** Phù hợp khi cần tìm chính xác một tài liệu cụ thể.

Nhược điểm:

- **Không có xếp hạng:** Tất cả tài liệu liên quan có cùng điểm, không phân biệt được tài liệu nào liên quan hơn.
- **Khó viết truy vấn:** Người dùng cần hiểu cách sử dụng toán tử logic.
- **Nhạy cảm với cách viết:** Kết quả có thể quá nhiều (Recall cao, Precision thấp)

hoặc quá ít (Recall thấp, Precision cao) tùy thuộc vào truy vấn.

2.3.2 TF-IDF

TF-IDF (Term Frequency - Inverse Document Frequency) là phương pháp thống kê nhằm đánh giá tầm quan trọng của một từ đối với một tài liệu trong một tập hợp tài liệu [1]. TF-IDF dựa trên hai yếu tố chính:

1. **Term Frequency (TF) - Tần suất xuất hiện:** Tần suất xuất hiện của từ trong văn bản. Ý tưởng là từ xuất hiện nhiều lần trong tài liệu thì quan trọng hơn.
2. **Inverse Document Frequency (IDF) - Nghịch đảo tần suất tài liệu:** Mức độ hiếm của từ đó trong toàn bộ tập tài liệu. Từ càng hiếm thì IDF càng cao, nghĩa là từ đó phân biệt tốt hơn giữa các tài liệu.

Một từ quan trọng là từ xuất hiện nhiều trong một tài liệu cụ thể (TF cao) nhưng lại ít xuất hiện trong các tài liệu khác (IDF cao).

Các công thức của TF-IDF:

Term Frequency (TF):

$$TF(t,d) = \frac{\text{Số lần xuất hiện của từ } t \text{ trong văn bản } d}{\text{Tổng số từ trong văn bản } d}$$

Hoặc công thức logarithm để giảm ảnh hưởng của tần suất lớn:

$$TF(t,d) = 1 + \log(\text{Số lần xuất hiện của từ } t \text{ trong văn bản } d)$$

Inverse Document Frequency (IDF):

$$IDF(t,D) = \log\left(\frac{\text{Tổng số văn bản}}{\text{Tổng số văn bản có chứa từ } t}\right)$$

Công thức có smoothing để tránh chia cho 0:

$$IDF(t,D) = \log\left(\frac{N+1}{n_t+1}\right) + 1$$

TF-IDF:

$$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$

Từ công thức này có thể chấm điểm độ liên quan của một tài liệu với truy vấn bằng cách cộng TF-IDF của các từ trong truy vấn.

Để đánh giá độ tương đồng giữa truy vấn và tài liệu, ta sử dụng độ đo cosine similarity [1]:

$$similarity(q, d) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} = \frac{\sum_{i=1}^{n} q_i \times d_i}{\sqrt{\sum_{i=1}^{n} q_i^2} \times \sqrt{\sum_{i=1}^{n} d_i^2}}$$

Trong đó $\vec{q}$ và $\vec{d}$ là vector biểu diễn truy vấn và tài liệu trong không gian nhiều chiều, mỗi chiều tương ứng một từ trong từ điển.

Ưu điểm của TF-IDF:

- **Đơn giản:** Dễ hiểu và dễ cài đặt.
- **Hiệu quả:** Tính toán nhanh, phù hợp với tập dữ liệu vừa và lớn.
- **Xếp hạng:** Cung cấp điểm liên quan để xếp hạng tài liệu.

Nhược điểm:

- **Giả định độc lập giữa các từ:** Không xử lý được mối quan hệ giữa các từ.
- **Không xử lý từ đồng nghĩa:** "Máy tính" và "computer" được coi là hai từ khác nhau.
- **Không xử lý từ đồng âm:** "Bank" (ngân hàng) và "bank" (bờ sông) được coi là cùng một từ.
- **Nhạy cảm với độ dài tài liệu:** Tài liệu dài có xu hướng có TF cao hơn.

2.3.3 BM25

BM25 (Best Matching 25) là một biến thể nâng cao của mô hình TF-IDF, thuộc họ các mô hình truy hồi thông tin dựa trên xác suất (Probabilistic Relevance Framework) [2]. BM25 được phát triển bởi Stephen Robertson và các cộng sự vào những năm 1970-1990 và đã trở thành một trong những phương pháp chấm điểm phổ biến nhất trong các hệ thống tìm kiếm.

Công thức BM25 để tính điểm của một tài liệu D đối với truy vấn Q (chứa các từ khóa $q_1, q_2, ..., q_n$):

$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Trong đó:

- $f(q_i, D)$: Tần suất xuất hiện của từ khóa q_i trong tài liệu D (raw term frequency).

- $|D|$: Độ dài của tài liệu D (tổng số từ trong tài liệu).
- avgdl: Độ dài trung bình của tất cả các tài liệu trong tập dữ liệu.
- k_1: Một tham số tự do (thường chọn từ 1.2 đến 2.0, mặc định 1.5). Nó kiểm soát mức độ ảnh hưởng của tần suất từ khóa (TF saturation).
- b: Một tham số tự do (thường là 0.75). Nó quyết định mức độ ảnh hưởng của độ dài tài liệu (length normalization).
- IDF(q_i): Trọng số nghịch đảo của tần suất tài liệu, thường được tính bằng công thức Okapi:

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$$

  Với N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa từ q_i.

Hai tham số quan trọng k_1 và b kiểm soát các đặc tính của BM25:

1. **TF Saturation (Bão hòa TF):** Khi TF tăng, điểm số tăng nhưng với tốc độ giảm dần (diminishing returns). Được kiểm soát bởi k_1.
   - $k_1 = 0$: TF không ảnh hưởng đến điểm số (giống như binary).
   - $k_1 \to \infty$: TF ảnh hưởng tuyến tính (giống như TF-IDF thuần túy).
2. **Length Normalization (Chuẩn hóa độ dài):** Chuẩn hóa theo độ dài document, được kiểm soát bởi b.
   - $b = 0$: Không chuẩn hóa theo độ dài.
   - $b = 1$: Chuẩn hóa hoàn toàn theo độ dài.
3. **IDF:** Từ hiếm có IDF cao $\to$ đóng góp lớn hơn vào điểm số.

Các giá trị mặc định $k_1 = 1.5$ và $b = 0.75$ được khuyến nghị dựa trên nhiều thực nghiệm trên các tập dữ liệu khác nhau [2]. Tuy nhiên, các giá trị tối ưu có thể khác nhau tùy thuộc vào tập dữ liệu và loại truy vấn.

Ưu điểm của BM25:

- **TF Saturation:** Tránh tình trạng tài liệu có từ khóa xuất hiện quá nhiều lần chiếm ưu thế.
- **Length Normalization:** Công bằng giữa tài liệu dài và ngắn.
- **Hiệu quả được chứng minh:** Là một trong những phương pháp hiệu quả nhất trong các tham chiếu đánh giá TREC.
- **Ít tham số:** Chỉ có 2 tham số cần điều chỉnh (k_1 và b).

Nhược điểm:

- **Không xử lý ngữ nghĩa:** Vẫn dựa trên từ khóa chính xác.
- **Không xử lý từ đồng nghĩa:** "Máy tính" và "computer" vẫn là hai từ khác nhau.
- **Không xử lý thứ tự từ:** Không xem xét vị trí của các từ trong câu.

BM25 đã trở thành tiêu chuẩn trong nhiều hệ thống tìm kiếm, bao gồm cả Elasticsearch.

2.4 Các thước đo đánh giá

Với cách đánh giá dựa trên thứ hạng (rank-based evaluation), đề tài sử dụng một số thước đo phổ biến để đánh giá chất lượng của các mô hình truy xuất [1].

2.4.1 Precision@K (Precision at K)

Precision at K (P@K) là tỷ lệ giữa số lượng tài liệu liên quan được tìm thấy trong K kết quả đầu tiên chia cho giá trị K. Chỉ số này trả lời cho câu hỏi: "Trong số K kết quả mà hệ thống trả về, có bao nhiêu phần trăm thực sự hữu ích?"

Công thức:

$$\text{Precision@}K = \frac{\#\{\text{tài liệu liên quan trong } K \text{ kết quả đầu tiên}\}}{K}$$

Đặc điểm của thước đo:

- Chỉ quan tâm đến tập K kết quả hàng đầu, không quan tâm đến các tài liệu liên quan nằm ngoài vị trí K.
- Dễ hiểu và trực quan đối với người dùng cuối (ví dụ: trang đầu tiên của Google thường hiển thị $K = 10$ kết quả).
- Thường được sử dụng trong các ứng dụng thực tế nơi người dùng chỉ xem một số lượng nhỏ kết quả.

Nhược điểm:

- Không tính đến thứ tự của các tài liệu liên quan bên trong tập K (ví dụ: tài liệu liên quan nằm ở vị trí thứ 1 hay thứ 10 thì P@10 vẫn như nhau).
- Không phản ánh đầy đủ chất lượng khi số tài liệu liên quan thực tế nhỏ hơn K.

2.4.2 Recall at K (Recall at K)

Recall at K (R@K) là tỷ lệ giữa số lượng tài liệu liên quan được tìm thấy trong K kết quả đầu tiên chia cho tổng số tài liệu liên quan thực tế có trong toàn bộ cơ sở dữ liệu. Chỉ số này trả lời cho câu hỏi: "Hệ thống đã tìm thấy được bao nhiêu phần trăm tài liệu tốt trong tổng số tài liệu tốt hiện có chỉ với K kết quả đầu tiên?"

Công thức:

$$\text{Recall@}K = \frac{\#\{\text{tài liệu liên quan được tìm thấy trong } K \text{ kết quả đầu tiên}\}}{\#\{\text{tổng số tài liệu liên quan trong toàn bộ cơ sở dữ liệu}\}}$$

Đặc điểm:

- R@K luôn tăng hoặc giữ nguyên khi K tăng.
- Rất quan trọng trong các hệ thống mà việc bỏ sót thông tin là nghiêm trọng (ví dụ: tìm kiếm hồ sơ y tế hoặc bằng sáng chế).
- Trong thực tế, thường có sự đánh đổi giữa Precision và Recall.

Nhược điểm:

- Không phản ánh chất lượng xếp hạng - chỉ cần đưa tất cả tài liệu liên quan vào top K (có thể rất lớn) để đạt Recall cao.

2.4.3 F1 at K (F1 at K)

F1 at K (F1@K) là trung bình điều hòa (harmonic mean) giữa Precision@K và Recall@K. Nó giúp cân bằng giữa việc tìm đúng (Precision) và tìm đủ (Recall) trong phạm vi K kết quả đầu tiên:

$$F1@K = 2 \cdot \frac{P@K \times R@K}{P@K + R@K}$$

F1@K đạt giá trị cao khi cả Precision@K và Recall@K đều cao, và là thước đo cân bằng hơn so với chỉ sử dụng một trong hai.

2.4.4 Mean Reciprocal Rank (MRR)

MRR là giá trị trung bình của các Reciprocal Rank (RR - nghịch đảo của thứ hạng) trên một tập hợp các truy vấn. MRR tập trung vào vị trí của tài liệu liên quan đầu tiên.

Công thức tính Reciprocal Rank (RR) cho từng truy vấn i:

$$RR_i = \frac{1}{rank_i}$$

Trong đó $rank_i$ là vị trí của tài liệu liên quan đầu tiên được tìm thấy cho truy vấn đó. Nếu không có tài liệu liên quan nào được tìm thấy, $RR_i = 0$.

Mean Reciprocal Rank (MRR):

$$MRR = \frac{1}{N}\sum_{i=1}^{N} RR_i = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{rank_i}$$

Đặc điểm của MRR:

- **Ưu tiên vị trí dẫn đầu:** MRR cực kỳ nhạy bén với việc tài liệu đúng nằm ở vị trí thứ 1 hay thứ 2. Ví dụ: vị trí thứ 1 cho 1 điểm, nhưng vị trí thứ 2 chỉ còn 0.5 điểm.
- **Phù hợp với tìm kiếm thực tế:** Rất phù hợp cho các hệ thống mà người dùng chỉ cần tìm thấy một kết quả đúng duy nhất (như tìm kiếm câu trả lời cho một câu hỏi cụ thể).
- **Đơn giản:** Chỉ cần biết vị trí của tài liệu liên quan đầu tiên.

Hạn chế:

- MRR chỉ quan tâm đến tài liệu liên quan đầu tiên. Nếu một danh sách có nhiều tài liệu liên quan khác ở phía sau, MRR sẽ hoàn toàn bỏ qua chúng.
- Không phản ánh chất lượng xếp hạng của toàn bộ danh sách.

MRR được sử dụng rộng rãi trong các tham chiếu như TREC, MQAP, v.v.

2.4.5 Mean Average Precision (MAP)

MAP là trung bình cộng của Average Precision (AP) trên tất cả các truy vấn. AP đánh giá toàn diện hơn MRR vì nó tính đến vị trí của tất cả các tài liệu liên quan, không chỉ tài liệu đầu tiên [1].

Average Precision (AP) được tính bằng trung bình cộng của độ chính xác tại mỗi vị trí mà tài liệu liên quan xuất hiện:

$$AP = \frac{1}{|R|}\sum_{i=1}^{n} P(i) \times rel(i)$$

Trong đó:

- $|R|$: Tổng số tài liệu liên quan trong toàn bộ cơ sở dữ liệu.
- $P(i)$: Precision tại vị trí i (tức là P@i).
- $rel(i)$: 1 nếu tài liệu tại vị trí i liên quan, 0 nếu không.

Hoặc viết gọn hơn:

$$AP = \frac{1}{|R|} \times \sum_{k:rank_k \text{ liên quan}} \frac{k}{\text{rank}_k}$$

Mean Average Precision (MAP):

$$MAP = \frac{1}{Q} \sum_{q=1}^{Q} AP_q$$

Đặc điểm của MAP:

- **Đánh giá toàn diện:** Tính đến vị trí của tất cả các tài liệu liên quan.
- **Xếp hạng cao cho hệ thống tốt:** Hệ thống đưa tài liệu liên quan lên đầu sẽ có MAP cao.
- **Nhạy cảm với thứ tự:** Phản ánh chính xác chất lượng xếp hạng.

MAP đánh giá toàn diện hơn MRR vì nó tính đến vị trí của tất cả các tài liệu liên quan, khng chỉ tài liệu đầu tiên.

2.5 Elasticsearch

Elasticsearch là một công cụ tìm kiếm và phân tích phân tán mạnh mẽ, được xây dựng trên nền tảng Apache Lucene - một thư viện tìm kiếm và truy xuất thông tin mã nguồn mở với hơn 15 năm kinh nghiệm [3]. Elasticsearch cung cấp một bộ máy tìm kiếm phân tán, có đầy đủ công cụ với giao diện web HTTP và hỗ trợ dữ liệu JSON. Được phát triển bằng Java và phát hành theo giấy phép Apache 2.0, Elasticsearch là một CSDL kiểu NoSQL với nhiệm vụ chính là lưu trữ và truy vấn tài liệu.

Elasticsearch đã trở thành một trong những công cụ tìm kiếm phổ biến nhất trong các ứng dụng doanh nghiệp nhờ các đặc điểm:

- **Phân tán và mở rộng:** Dễ dàng mở rộng theo chiều ngang bằng cách thêm node.
- **Tìm kiếm full-text:** Hỗ trợ tìm kiếm toàn văn với nhiều tính năng nâng cao.
- **Real-time:** Dữ liệu được đánh chỉ mục và có thể truy vấn ngay lập tức.
- **RESTful API:** Giao diện HTTP/JSON dễ sử dụng với nhiều ngôn ngữ lập trình.
- **Hệ sinh thái:** Nhiều tool và plugin hỗ trợ (Kibana, Logstash, Beats).

2.5.1 Các thành phần chính

a, Tài liệu (Document)

Một tài liệu (Document) là bản ghi thông tin nhỏ nhất được lưu bởi Elasticsearch và được biểu diễn dưới dạng JSON. Mỗi tài liệu có một ID duy nhất và chứa dữ liệu của các trường (fields) dưới dạng cặp khóa-giá trị (key-value). Elasticsearch hỗ trợ nhiều kiểu dữ liệu như text, keyword, number, date, boolean, geo, v.v.

b, Chỉ mục (Index)

Một chỉ mục (Index) là tập hợp logical của các document có cùng schema (mặc định). Mỗi index có một tên duy nhất và có thể được chia thành nhiều shard.

c, Shard

Các chỉ mục của Elasticsearch được chia thành các đơn vị có thể quản lý gọi là shard, là tập hợp các document. Shard là đơn vị cơ bản của tìm kiếm và có thể được sao chép (replica) trên nhiều node để dự phòng và chịu lỗi.

d, Node

Một node là một máy ảo (instance) độc lập của Elasticsearch, quản lý một tập hợp các shard thuộc về một hoặc nhiều chỉ mục. Node có thể có các vai trò khác nhau:

- **Master node:** Quản lý cluster, metadata.
- **Data node:** Lưu trữ và xử lý dữ liệu.
- **Ingest node:** Tiền xử lý dữ liệu trước khi đánh chỉ mục.
- **Coordinating node:** Tiếp nhận yêu cầu và điều phối.

e, Cụm (Cluster)

Một cụm (Cluster) là tập hợp các node được kết nối với nhau, làm việc cùng nhau để cung cấp khả năng tìm kiếm và lưu trữ phân tán.

2.5.2 Cách Elasticsearch đánh chỉ mục

Elasticsearch sử dụng chỉ mục ngược (inverted index) để đánh chỉ mục, tương tự như đã mô tả ở phần trước. Tuy nhiên, Elasticsearch bổ sung thêm nhiều tính năng nâng cao như:

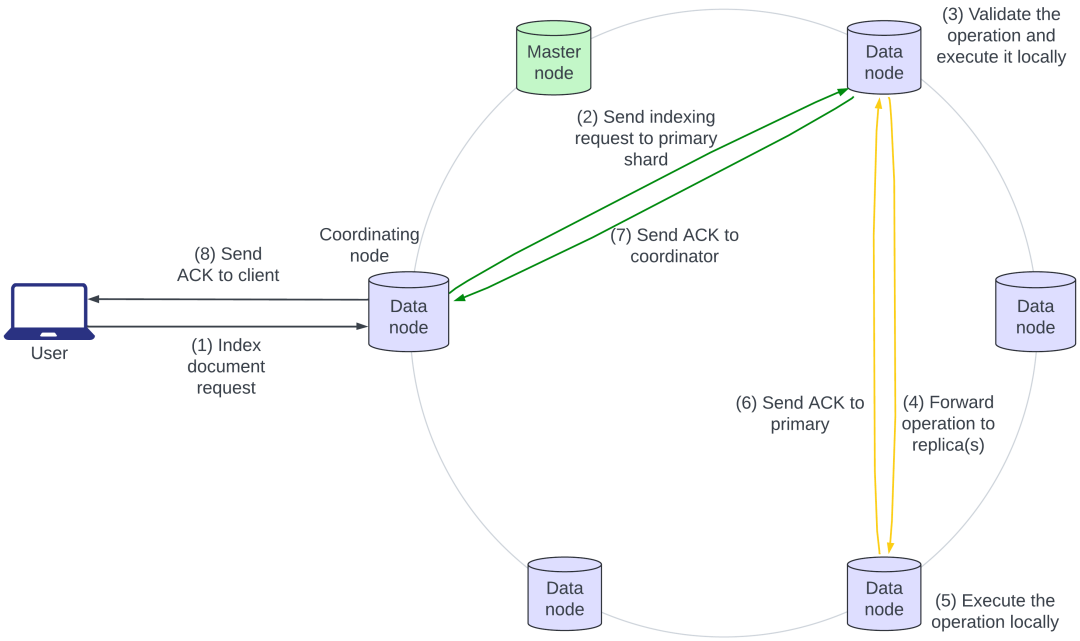
- **Phân tích văn bản (Analysis):** Tự động tách từ, chuẩn hóa, và tạo inverted index.
- **BM25 scoring:** Sử dụng BM25 làm thước đo mặc định để đánh giá liên quan.
- **Real-time indexing:** Dữ liệu mới được đánh chỉ mc ngay lập tức (sau 1 giây).

- **Compressed storage:** Lưu trữ hiệu quả với nén dữ liệu.

Quá trình đánh chỉ mục trong Elasticsearch:

1. Người dùng gửi một tài liệu dưới dạng JSON đến Elasticsearch. Node nhận yêu cầu đầu tiên là "node điều phối" (coordinating node).
2. Node điều phối xác định primary shard của tài liệu (dựa trên ID hoặc hash) và chuyển tiếp yêu cầu đến data node sở hữu primary shard đó.
3. Primary shard xác thực và thực thi thao tác ghi tại chỗ.
4. Primary shard sau đó chuyển tiếp thao tác đến tất cả replica shard một cách song song.
5. Các replica shard áp dụng thao tác và trả về acknowledgment.
6. Khi đủ số replication đạt yêu cầu, primary trả về kết quả thành công cho node điều phối và cho người dùng.

Hình 2.2 minh họa các bước lập chỉ mục trong Elasticsearch.



Hình 2.2: Các bước lập chỉ mục trong Elasticsearch

2.5.3 Cách Elasticsearch truy vấn

a, BM25 - Thước đo mặc định

Elasticsearch sử dụng BM25 làm thước đo mặc định để đánh giá xếp hạng mức độ liên quan của truy vấn với tài liệu. BM25 trong Elasticsearch sử dụng các tham số mặc định $k_1 = 1.2$ và $b = 0.75$, đã được chứng minh hiệu quả trên nhiều tập dữ liệu.

b, Các loại truy vấn

Elasticsearch hỗ trợ nhiều loại truy vấn:

- **Match Query:** Tìm kiếm full-text với phân tích token.
- **Term Query:** Tìm chính xác một giá trị (không qua phân tích).
- **Bool Query:** Kết hợp nhiều truy vấn bằng các mệnh đề must, should, must_not, filter.
- **Range Query:** Tìm trong một khoảng giá trị.
- **Wildcard Query:** Tìm với ký tự đại diện.

c, Quá trình truy vấn

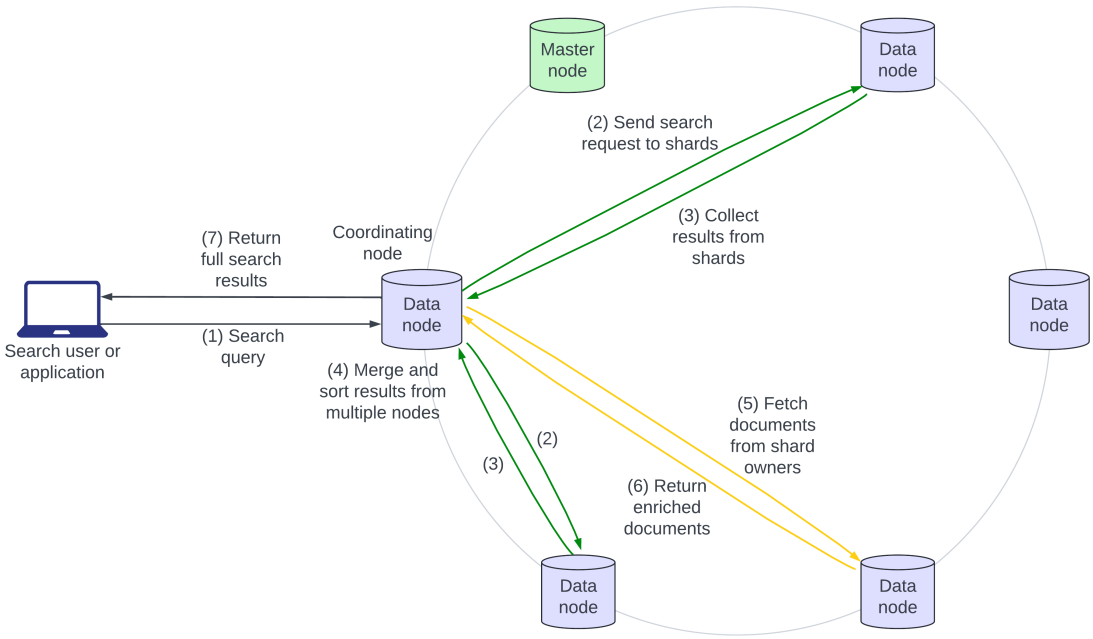
Quá trình truy vấn trong Elasticsearch gồm các pha:

1. **Pha truy vấn (Query phase):** Yêu cầu được phát tới các shard liên quan.
2. **Pha tính điểm (Scoring phase):** Mỗi shard tính điểm liên quan của tài liệu.
3. **Pha tổng hợp (Fetch phase):** Kết quả được hợp nhất và xếp hạng.

Chi tiết các bước truy vấn:

1. Người dùng hoặc ứng dụng gửi một truy vấn tìm kiếm. Truy vấn có thể được xử lý bởi bất kỳ node nào trong cụm (coordinating node).
2. Node điều phối phát broadcast truy vấn đến tất cả các shard liên quan và các replica.
3. Mỗi shard thực thi truy vấn cục bộ và trả về một tập kết quả lightweight (chỉ có ID và điểm) cho node điều phối.
4. Node điều phối hợp nhất các kết quả và sắp xếp theo điểm.
5. Node điều phối gửi yêu cầu truy xuất tài liệu đầy đủ đến các shard sở hữu.
6. Các tài liệu đầy đủ được trả về cho node điều phối.
7. Kết quả được trả về cho người dùng.

Hình 2.3 minh họa các bước truy vấn trong Elasticsearch.



Hình 2.3: Các bước truy vấn trong Elasticsearch

d, Xử lý ngôn ngữ

Elasticsearch hỗ trợ phân tích và tách từ đa ngôn ngữ. Tuy nhiên, bộ phân tích mặc định (standard analyzer) được thiết kế cho tiếng Anh. Với tiếng Việt, cần sử dụng bộ phân tích tùy chỉnh hoặc plugin hỗ trợ tiếng Việt.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

3.1 Tập dữ liệu

Tập dữ liệu là tập gồm các văn bản luật Việt Nam, các câu hỏi và danh sách các tài liệu liên quan cho câu hỏi đó. Dữ liệu này được đăng tải trên trang Huggingface [4], là một nền tảng chia sẻ tập dữ liệu machine learning phổ biến. Tập dữ liệu này được xây dựng nhằm mục đích phục vụ cho bài toán truy xuất thông tin pháp luật Việt Nam, một ứng dụng quan trọng trong việc tìm kiếm và tra cứu văn bản pháp luật một cách nhanh chóng và chính xác.

Các văn bản được đánh chỉ mục và xếp hạng dựa trên TF-IDF và BM25. Trong đánh giá, nhóm sử dụng kết hợp cả tập huấn luyện và tập kiểm thử, lấy 2000 mẫu đầu tiên (do giới hạn về phần cứng thiết bị) để chạy đánh giá và lấy kết quả. Việc giới hạn này là cần thiết do tập dữ liệu gốc có kích thước rất lớn, việc xử lý toàn bộ sẽ tốn nhiều thời gian và tài nguyên tính toán.

3.1.1 Tập văn bản

Tập văn bản (corpus) là tập hợp các văn bản pháp luật Việt Nam, được lưu trong file `corpus.csv` với 2 cột chính:

- **cid**: ID duy nhất của mỗi văn bản, dùng để định danh và truy xuất.
- **text**: Nội dung đầy đủ của đoạn hoặc điều luật, là văn bản thô cần được xử lý.

Theo thống kê, có tổng cộng 222.981 tài liệu trong corpus, mỗi tài liệu là một đoạn văn bản pháp luật (có thể là một điều, khoản, điểm hoặc một đoạn văn bản trong các văn bản luật). Đây là một tập dữ liệu tương đối lớn, phản ánh khối lượng văn bản pháp luật khổng lồ của hệ thống pháp luật Việt Nam. Mỗi tài liệu thường có độ dài ngắn, tập trung vào một điều khoản cụ thể, điều này phù hợp cho việc truy xuất thông tin khi người dùng tìm kiếm một quy định cụ thể.

Đặc điểm của tập dữ liệu:

- **Tính chuyên ngành cao:** Các văn bản sử dụng ngữ pháp và thuật ngữ pháp luật chuyên ngành.
- **Cấu trúc rõ ràng:** Các văn bản được phân chia theo điều, khoản, điểm.
- **Ngôn ngữ tiếng Việt:** Toàn bộ văn bản bằng tiếng Việt với đầy đủ dấu.
- **Độ dài ngắn:** Phần lớn các tài liệu có độ dài ngắn (trung bình 20-50 từ).

3.1.2 Tập dữ liệu huấn luyện và tập dữ liệu kiểm thử

Do đề tài sử dụng các phương pháp đánh giá dựa trên điểm (score-based) không phải huấn luyện mô hình máy học, do đó để đánh giá mô hình nhóm sử dụng kết hợp cả hai tập dữ liệu. Tuy nhiên do giới hạn phần cứng thiết bị, nhóm chỉ sử dụng 2000 mẫu đầu tiên để tiến hành đánh giá.

Tập dữ liệu có cấu trúc các cột như sau:

Cột	Kiểu	Mô tả
`qid`	int	ID câu hỏi duy nhất
`question`	string	Câu hỏi tìm kiếm
`cid`	string (list)	Danh sách ID văn bản liên quan
`context_list`	string (list)	Nội dung đoạn văn liên quan

Tổng số câu hỏi của tập dữ liệu huấn luyện là 9.541 và tập dữ liệu kiểm thử là 3.184. Nhóm lấy 2.000 mẫu đầu tiên để làm nhiệm vụ đánh giá hệ thống truy xuất thông tin. Trong toàn bộ các tài liệu, mỗi truy vấn chỉ có 1 hoặc một vài tài liệu liên quan. Điều này cho thấy tỉ lệ mẫu dương tính (positive) rất thấp - đây là thách thức lớn cho các hệ thống truy xuất thông tin vì:

- Hệ thống phải tìm một "kim đốt giữa mo" trong hàng trăm nghìn tài liệu.
- Precision@K sẽ bị ảnh hưởng nghiêm trọng khi K nhỏ.
- Recall@10 đạt 77% là kết quả rất tốt trong điều kiện này.

Đặc điểm của tập dữ liệu đánh giá:

- **Tài liệu liên quan thưa thớt:** Mỗi truy vấn chỉ có vài tài liệu liên quan.
- Các câu hỏi được viết bằng ngôn ngữ tự nhiên.
- Bao gồm nhiều lĩnh vực pháp luật khác nhau.
- Có sẵn danh sách tài liệu liên quan làm ground truth.

Để hiểu rõ hơn về độ khó của bài toán, ta có thể phân tích tỷ lệ tài liệu liên quan trên tổng số tài liệu. Với tổng cộng 222.981 tài liệu trong corpus và trung bình chỉ có khoảng 1-3 tài liệu liên quan cho mỗi truy vấn, tỷ lệ này dao động từ 0.0004% đến 0.0013%. Đây là một tỷ lệ cực kỳ thấp, đặt ra thách thức lớn cho các thuật toán truy xuất thông tin. Hệ thống cần phải tìm chính xác những tài liệu thuộc phần nhỏ bé này trong hàng trăm nghìn tài liệu khác.

Một đặc điểm quan trọng khác của tập dữ liệu là sự đa dạng về chủ đề pháp lý. Các câu hỏi trong tập dữ liệu bao phủ nhiều lĩnh vực như luật hình sự, luật dân sự, luật hành chính, luật thương mại, luật đất đai, luật lao động, v.v. Điều này đảm bảo

rằng kết quả đánh giá không bị thiên lệch về một lĩnh vực cụ thể mà phản ánh hiệu suất tổng thể của hệ thống trên nhiều loại truy vấn khác nhau.

3.2 Xây dựng chỉ mục Elasticsearch

Elasticsearch là một công cụ tìm kiếm phân tán mạnh mẽ, được sử dụng rộng rãi trong các ứng dụng thực tế. Trong đề tài này, Elasticsearch đóng vai trò là baseline để so sánh với các mô hình cài đặt thủ công. Việc sử dụng Elasticsearch làm baseline mang lại nhiều lợi ích: thứ nhất, đây là một hệ thống tìm kiếm đã được tối ưu hóa và chứng minh hiệu quả trong nhiều ứng dụng công nghiệp; thứ hai, Elasticsearch cung cấp nhiều tính năng mạnh mẽ như tìm kiếm full-text, phân tích ngôn ngữ tự nhiên, và khả năng mở rộng quy mô; thứ ba, việc so sánh với Elasticsearch giúp đánh giá xem các cài đặt thủ công có thể cạnh tranh được với các giải pháp sẵn có hay không.

Trong đề tài này, nhóm sử dụng phiên bản Elasticsearch 8.x với Python client tương ứng. Cấu hình cluster được thiết kế đơn giản phù hợp với môi trường thử nghiệm: một node master-dual được cấu hình với 2GB heap memory. Mặc dù đây là cấu hình nhỏ, nó đủ để xử lý tập dữ liệu 222.981 văn bản pháp lý một cách hiệu quả.

Hai phương pháp đánh chỉ mục

Với bước xây dựng chỉ mục, đề tài sử dụng 2 cách để đánh giá và so sánh:

1. **ES với chỉ mục mặc định (Normal index):** Sử dụng bộ phân tích (analyzer) mặc định của Elasticsearch, được thiết kế cho tiếng Anh. Bộ phân tích này sẽ tách từ theo dấu cách và dấu câu.
2. **ES với chỉ mục đã xử lý (Processed index):** Tiến hành tách từ tiếng Việt bằng thư viện underthesea trước, sau đó nối lại theo dấu cách và đưa vào chỉ mục của Elasticsearch.

Sự khác biệt giữa hai phương pháp:

- **Chỉ mục bình thường:** Elasticsearch tự động tokenize và phân tích văn bản.
- **Chỉ mục đã xử lý:** Văn bản đã được underthesea xử lý trước khi đưa vào Elasticsearch.

Mục tiêu của việc so sánh này là chứng minh tầm quan trọng của tiền xử lý văn bản tiếng Việt trong hệ thống tìm kiếm. Khi sử dụng bộ phân tích mặc định của Elasticsearch, văn bản tiếng Việt sẽ bị tách từ một cách không chính xác do đặc điểm của tiếng Việt là không có dấu cách giữa các từ trong từ ghép. Ví dụ, câu "Người phạm tội sẽ bị truy cứu trách nhiệm hình sự" sẽ bị tách thành các token

riêng lẻ như "Người", "phạm", "tội", "sẽ", "bị", "truy", "cứu", "trách", "nhiệm", "hình", "sự" thay vì các từ ghép có nghĩa như "Người", "phạm", "tội", "truy", "cứu", "trách", "nhiệm", "hình", "sự".

Điều này dẫn đến việc tìm kiếm với từ khóa "truy cứu" sẽ không khớp với văn bản vì Elasticsearch đã tách "truy" và "cứu" thành hai token riêng biệt. Ngược lại, khi văn bản đã được xử lý trước bằng underthesea, từ ghép "truy_cứu" được giữ nguyên và việc tìm kiếm sẽ chính xác hơn.

Kết quả thực nghiệm cho thấy chỉ mục đã xử lý cho kết quả tốt hơn, chứng tỏ tiền xử lý văn bản tiếng Việt là cần thiết.

Quy trình đánh chỉ mục được thực hiện như sau:

1. **Đọc dữ liệu:** Đọc file corpus.csv chứa 222.981 văn bản pháp lý.
2. **Xử lý trước (cho Processed index):** Với mỗi văn bản, sử dụng underthesea để tách từ và nối từ ghép bằng dấu "_".
3. **Tạo index:** Tạo index trong Elasticsearch với mapping định nghĩa trước.
4. **Index documents:** Đưa từng văn bản vào index với các trường cid và content.
5. **Refresh:** Refresh index để các document có thể được tìm kiếm ngay lập tức.

Thời gian đánh chỉ mục cho toàn bộ 222.981 văn bản là khoảng 5-10 phút tùy thuộc vào cấu hình phần cứng. Kích thước index sau khi đánh chỉ mục là khoảng 150-200 MB, bao gồm dữ liệu văn bản và các cấu trúc phục vụ tìm kiếm.

Cấu hình Mapping

Mapping là cấu hình cấu trúc dữ liệu cho index trong Elasticsearch. Mapping định nghĩa cách dữ liệu được lưu trữ và index, bao gồm kiểu dữ liệu, cách phân tích, và các thuộc tính khác. Với trường content trong dữ liệu, kiểu dữ liệu khi ánh xạ vào chỉ mục của Elasticsearch là `text`.

```
mapping = {
    "mappings": {
        "properties": {
            "cid": {"type": "keyword"},
            "content": {
                "type": "text",
                "analyzer": "standard",
                "fields": {
                    "keyword": {"type": "keyword"}
                }
            }
        }
```

```
13        }
14 }
```

Elasticsearch sẽ phân tích, phân tách nội dung thành các token riêng lẻ và phù hợp cho việc tìm kiếm tài liệu chứa các từ. Tuy nhiên, trường kiểu `text` không thể dùng cho sắp xếp, tổng hợp hoặc tìm kiếm chính xác tuyệt đối (exact match).

Phần tiếp theo của cấu hình mapping là phần analyzer, sử dụng bộ phân tích chuẩn (standard analyzer). Bộ phân tích này của Elasticsearch gồm 3 thành phần:

1. **Tokenizer:** Tách văn bản thành từng token theo dấu cách, dấu câu.
2. **Lowercase filter:** Chuyển tất cả các ký tự về chữ thường.
3. **Stop Token Filter:** Loại bỏ từ dừng tiếng Anh mặc định.

Bộ phân tích standard được thiết kế chủ yếu cho tiếng Anh, do đó khi áp dụng cho tiếng Việt, nó chỉ có thể tách từ dựa trên dấu cách. Điều này không phù hợp với đặc điểm của tiếng Việt - ngôn ngữ sử dụng dấu cách để phân tách từ trong câu, nhưng các từ ghép không có dấu cách giữa các tiếng. Vì vậy, để cải thiện hiệu quả tìm kiếm cho tiếng Việt, cần sử dụng các phương pháp tiền xử lý như tách từ bằng underthesea trước khi đưa vào Elasticsearch.

Phần tiếp theo là trường keywords - tạo thêm một trường con `content.keyword` bên cạnh `content`. Bảng 3.1 so sánh giữa 2 trường content và content.keyword.

Thuộc tính	content (text)	content.keyword (keyword)
Loại	Full-text	Exact match
Phân tích	Có (tokenize)	Không (giữ nguyên)
Dùng cho	Match query, search	Sắp xếp, tổng hợp, lọc
Ví dụ	Tìm một từ trong văn bản	So sánh chính xác chuỗi

Bảng 3.1: So sánh trường `content` và trường `content.keyword`

Khi phân tích tác động đến kết quả đánh giá, dễ thấy rằng với ngôn ngữ tiếng Việt, chỉ mục đã được tách từ sử dụng thư viện underthesea cho kết quả tốt hơn khi sử dụng cách tách từ chuẩn của Elasticsearch. Điều này chứng tỏ ngôn ngữ mặc định áp dụng của Elasticsearch là tiếng Anh không phù hợp với tiếng Việt, và việc tiền xử lý là cần thiết.

Để tối ưu hóa hiệu suất tìm kiếm, Elasticsearch còn hỗ trợ nhiều loại query khác nhau. Trong đề tài này, nhóm sử dụng `MultiMatchQuery` với trường `content` để thực hiện tìm kiếm full-text. Query này cho phép tìm kiếm một từ hoặc cụm từ trên nhiều trường cùng lúc và tính điểm dựa trên mức độ khớp.

3.3 Xây dựng chỉ mục ngược

3.3.1 Quy trình xây dựng chỉ mục ngược

Chỉ mục ngược được xây dựng từ tập dữ liệu văn bản pháp luật đã qua tiền xử lý. Quy trình xây dựng chỉ mục ngược bao gồm các bước sau:

Bước 1: Tạo cặp (term, docID)

Với mỗi tài liệu trong corpus, sau khi tách từ và loại bỏ từ dừng, tạo ra danh sách các cặp (term, docID). Ví dụ, nếu tài liệu Doc1 chứa các từ "thuế", "luật", "phạt", ta tạo ra các cặp: (thuế, Doc1), (luật, Doc1), (phạt, Doc1).

Bước 2: Sắp xếp

Sắp xếp tất cả các cặp (term, docID) theo thứ tự từ điển của term, sau đó theo docID. Độ phức tạp của bước này là $O(N \times L \times \log(N \times L))$, với N là số tài liệu và L là độ dài trung bình của tài liệu.

Bước 3: Gộp postings

Duyệt qua danh sách đã sắp xếp và gộp các mục trùng lặp. Với mỗi term, tạo postings list chứa danh sách các docID và tần suất xuất hiện. Cấu trúc dữ liệu được tổ chức dưới dạng từ điển với key là term, value chứa doc_freq (số tài liệu chứa term) và postings list (danh sách các cặp docID và term frequency).

Bước 4: Lưu trữ

Lưu chỉ mục ngược vào file để tái sử dụng mà không cần xây dựng lại. Chỉ mục được lưu dưới dạng JSON để dễ đọc và xử lý. Kích thước chỉ mục khoảng 150-200 MB cho 222,981 tài liệu.

3.3.2 Tối ưu hóa chỉ mục

Để tăng tốc độ truy vấn, một số tối ưu hóa được áp dụng:

- **Chỉ mục trong bộ nhớ:** Toàn bộ chỉ mục được nạp vào RAM khi khởi động hệ thống, giúp truy vấn nhanh mà không cần đọc từ đĩa.
- **Cấu trúc dữ liệu tối ưu:** Sử dụng dictionary của Python với độ phức tạp truy cập $O(1)$ cho việc tìm kiếm term.
- **Danh sách postings được sắp xếp:** Mỗi postings list được sắp xếp theo docID để tối ưu các phép toán tập hợp (AND, OR).

3.3.3 Các phép toán trên chỉ mục ngược

Các phép toán tập hợp trên chỉ mục ngược được thực hiện bằng thuật toán merge với độ phức tạp tuyến tính.

Phép AND: Tìm các tài liệu chứa cả hai từ.

Algorithm 1: Intersect (Phép AND)

```
Input: P_1, P_2 - hai danh sách postings đã sắp xếp
Output: R - danh sách kết quả
i ← 0, j ← 0, R ← [];
while i < |P_1| and j < |P_2| do
    if P_1[i] = P_2[j] then
        Thêm P_1[i] vào R;
        i ← i + 1, j ← j + 1;
    else
        if P_1[i] < P_2[j] then
            i ← i + 1;
        else
            j ← j + 1;
        end
    end
end
return R
```

Phép OR: Tìm các tài liệu chứa ít nhất một trong hai từ.

Algorithm 2: Union (Phép OR)

Input: P_1, P_2 - hai danh sách postings đã sắp xếp
Output: R - danh sách kết quả
$i \leftarrow 0$, $j \leftarrow 0$, $R \leftarrow []$;
while $i < |P_1|$ ***and*** $j < |P_2|$ **do**
  if $P_1[i] = P_2[j]$ **then**
    Thêm $P_1[i]$ vào R;
    $i \leftarrow i + 1$, $j \leftarrow j + 1$;
  else
    if $P_1[i] < P_2[j]$ **then**
      Thêm $P_1[i]$ vào R, $i \leftarrow i + 1$;
    else
      Thêm $P_2[j]$ vào R, $j \leftarrow j + 1$;
    end
  end
end
Nối phần còn lại của P_1 hoặc P_2 vào R;
return R

Phép NOT: Tìm các tài liệu không chứa từ.

Algorithm 3: Complement (Phép NOT)

Input: P - danh sách postings của từ cần loại, D - tập tất cả docID
Output: R - danh sách kết quả
$R \leftarrow D$;
foreach $doc \in P$ **do**
  Loại bỏ doc khỏi R;
end
return R

3.4 Các phương pháp truy xuất

3.4.1 Mô hình Boolean

Mô hình Boolean được cài đặt dựa trên các phép toán tập hợp trên chỉ mục ngược. Mỗi truy vấn chỉ chứa tối đa một toán tử (AND, OR, hoặc NOT) để đơn giản hóa việc xử lý.

Cấu trúc truy vấn:

Truy vấn Boolean được biểu diễn dưới dạng danh sách các token, trong đó có thể chứa một toán tử (AND, OR, NOT) để phân tách hai nhóm term:

- `first_terms`: Các term trước toán tử
- `operator`: Toán tử AND, OR, hoặc NOT
- `second_terms`: Các term sau toán tử

Ví dụ: truy vấn "thuế AND luật" sẽ được tách thành first_terms = ["thuế"], operator = "AND", second_terms = ["luật"].

Thuật toán tìm kiếm Boolean:

Algorithm 4: BooleanSearch

```
Input: query - danh sách các token truy vấn
Output: result - danh sách các docID
first_terms ← [], second_terms ← [], operator ← "" is_first ← true
foreach term ∈ query do
    if term ∈ {AND, OR, NOT} then
        is_first ← false operator ← term continue
    end
    if is_first then
        Thêm term vào first_terms
    end
    Thêm term vào second_terms
end
if operator = "" then
    operator ← AND;    /* Mặc định AND nếu không có toán
     tử */
end
switch operator do
    case AND do
        all_terms ← first_terms + second_terms
         result ← getDocsContaining(all_terms[0]) foreach
         term ∈ all_terms[1 :] do
            result ← result ∩ getDocsContaining(term)
        end
    end
    case OR do
        result ← ∅ foreach term ∈ first_terms do
            result ← result ∪ getDocsContaining(term)
        end
        foreach term ∈ second_terms do
            result ← result ∪ getDocsContaining(term)
        end
    end
    case NOT do
        all_docs ← getAllDocIds() excluded ← ∅ foreach
         term ∈ first_terms + second_terms do
            excluded ← excluded ∪ getDocsContaining(term)
        end
        result ← all_docs − excluded
    end
end
return result
```

Hạn chế: Do không có cơ chế xếp hạng, tất cả tài liệu khớp được trả về với cùng mức độ ưu tiên.

3.4.2 Mô hình không gian vector (VSM)

VSM được cài đặt với các bước sau:

Bước 1: Xây dựng không gian vector

Xác định không gian vector với số chiều bằng số từ duy nhất trong toàn bộ corpus (khoảng 50,000 chiều sau khi loại bỏ từ dừng). Mỗi tài liệu và truy vấn được biểu diễn dưới dạng vector trong không gian này.

Bước 2: Tính trọng số TF-IDF

Với mỗi từ t trong tài liệu d, trọng số TF-IDF được tính như sau:

Tần suất xuất hiện của từ t trong tài liệu d được tính bằng số lần xuất hiện của t chia cho tổng số từ trong d. Để giảm ảnh hưởng của các tài liệu dài, có thể sử dụng công thức logarit: $TF(t, d) = 1 + \log(\text{số lần xuất hiện của } t)$.

IDF của từ t được tính bằng logarit của tổng số tài liệu chia cho số tài liệu chứa t, với công thức smoothing để tránh chia cho 0: $IDF(t) = \log\left(\frac{N+1}{n_t+1}\right) + 1$, trong đó N là tổng số tài liệu và n_t là số tài liệu chứa từ t.

Trọng số TF-IDF cuối cùng là tích của TF và IDF.

Bước 3: Truy vấn

1. Tiền xử lý truy vấn: tách từ, loại bỏ từ dừng.
2. Tìm ứng viên: Sử dụng chỉ mục ngược để tìm tất cả tài liệu chứa ít nhất một từ trong truy vấn.
3. Tính độ tương đồng cosine: Với mỗi tài liệu ứng viên, tính tích vô hướng của vector truy vấn và vector tài liệu, chia cho tích của độ dài hai vector. Công thức: $similarity(q, d) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|}$.
4. Sắp xếp và trả về top-K tài liệu có độ tương đồng cao nhất.

Tối ưu hóa: Thay vì lưu toàn bộ ma trận TF-IDF (222,981 x 50,000), hệ thống chỉ tính toán TF-IDF cho các tài liệu ứng viên được tìm thấy qua chỉ mục ngược, giảm đáng kể chi phí tính toán.

3.4.3 Mô hình BM25

BM25 được cài đặt dựa trên công thức chuẩn với các tham số $k_1 = 1.5$ và $b = 0.75$.

Cấu trúc dữ liệu:

Các giá trị cần tính trước được lưu trong chỉ mục:

- $f(q_i, D)$: term frequency trong mỗi tài liệu
- $|D|$: độ dài mỗi tài liệu
- N: tổng số tài liệu
- avgdl: độ dài trung bình của corpus

Thuật toán tính điểm BM25:

Algorithm 5: ScoreBM25

Input: Q - truy vấn (danh sách các term), D - tài liệu

Output: $score$ - điểm số BM25

$score \leftarrow 0$;

foreach $q_i \in Q$ **do**

  if $q_i \notin D$ **then**

    ;

  end

  $f \leftarrow \text{getTF}(q_i, D)$;

  $n_q \leftarrow \text{getDocFreq}(q_i)$;

  $idf \leftarrow \ln\left(\frac{N-n_q+0.5}{n_q+0.5} + 1\right)$;

  $numerator \leftarrow f \times (k_1 + 1)$;

  $denominator \leftarrow f + k_1 \times \left(1 - b + b \times \frac{|D|}{\text{avgdl}}\right)$;

  $score \leftarrow score + idf \times \frac{numerator}{denominator}$;

end

return $score$

Thuật toán truy vấn BM25:

Algorithm 6: QueryBM25

Input: Q - chuỗi truy vấn, k - số kết quả cần trả về
Output: R - danh sách top-k tài liệu
$Q \leftarrow$ preprocess(Q);
candidates $\leftarrow$ getCandidates(Q);
scores $\leftarrow$ [];
foreach $D \in$ *candidates* **do**
  $s \leftarrow$ ScoreBM25(Q, D);
  scores.append$((D, s))$;
end
scores.sort(by score descending);
$R \leftarrow$ scores$[0 : k]$;
return R

So sánh với VSM: BM25 có hai cải tiến chính so với VSM:

- Độ bão hòa TF (TF saturation): Giới hạn đóng góp của tần suất cao.
- Chuẩn hóa độ dài (Length normalization): Cân bằng giữa tài liệu dài và ngắn.

Kết quả thực nghiệm cho thấy BM25 vượt trội hơn VSM đáng kể (MRR 0.5269 so với 0.3353).

3.5 Kết quả và đánh giá

Tiến hành đánh giá kết quả trên tập có 2000 truy vấn được lấy từ 2000 truy vấn đầu tiên khi kết hợp tập dữ liệu huấn luyện và tập kiểm thử.

Pipeline đánh giá được thực hiện như sau:

Algorithm 7: Pipeline đánh giá

Input: search_engine, ground_truth: Dict[qid, List[cid]], queries: Dict[qid, question]

1. **Với mỗi phương pháp:**
   Với mỗi method trong {Boolean, VSM, BM25, ES_Normal, ES_Processed}:
   (a) **Với mỗi truy vấn:**
   Với mỗi $(qid, question)$ trong queries:
   - Thực hiện tìm kiếm:
     $results \leftarrow search_engine.search(question, method, top_n = 10)$
   - Trích xuất ID: $retrieved_ids$, $true_ids$
   - Tính các chỉ số: P@K, R@K, F1@K, RR, AP cho mỗi K
2. **Tổng hợp:** Tính trung bình của tất cả các chỉ số

3.5.1 Kết quả đánh giá tổng quan

Bảng 3.2 trình bày kết quả đánh giá tổng quan của 5 phương pháp truy xuất trên các chỉ số Precision@1, Precision@5, Precision@10, Recall@10, F1@10, MRR và MAP.

Phương pháp	P@1	P@5	P@10	R@10	F1@10	MRR	MAP
ES (Đã xử lý)	**0.4132**	**0.1349**	**0.0771**	**0.7708**	**0.1401**	**0.5284**	**0.5284**
BM25	**0.4120**	**0.1358**	**0.0770**	**0.7695**	**0.1399**	**0.5269**	**0.5269**
ES (Bình thường)	0.4012	0.1339	0.0737	0.7372	0.1340	0.5136	0.5136
VSM	0.2195	0.0963	0.0627	0.6270	0.1140	0.3353	0.3353
Boolean	0.1305	0.0541	0.0306	0.3060	0.0556	0.1900	0.1900

Bảng 3.2: Kết quả đánh giá tổng quan các phương pháp

Từ bảng kết quả, có thể thấy rằng Elasticsearch với chỉ mục đã xử lý đạt hiệu suất cao nhất với MRR và MAP đạt 0.5284. BM25 đứng thứ hai với MRR 0.5269, chỉ kém ES Processsed 0.0015. Điều này cho thấy thuật toán BM25 cài đặt thủ công có thể cạnh tranh được với Elasticsearch - một hệ thống tìm kiếm thương mại đã được tối ưu hóa.

ES với chỉ mục bình thường đạt MRR 0.5136, thấp hơn 0.0148 so với ES đã xử lý. Sự khác biệt này chứng minh tầm quan trọng của tiền xử lý văn bản tiếng Việt trước khi đưa vào hệ thống tìm kiếm. Khi văn bản được tách từ bằng underthesea, Elasticsearch có thể tìm kiếm chính xác hơn các từ ghép tiếng Việt.

VSM đạt MRR 0.3353, thấp hơn đáng kể so với BM25 và Elasticsearch. Nguyên nhân chính là VSM sử dụng TF-IDF đơn giản mà không có các cơ chế bão hòa TF và chuẩn hóa độ dài tài liệu như BM25.

Mô hình Boolean có hiệu suất thấp nhất với MRR chỉ 0.1900. Điều này là do mô hình Boolean không có cơ chế xếp hạng - tất cả tài liệu khớp đều được coi là có cùng mức độ liên quan, dẫn đến việc tài liệu liên quan có thể bị chìm trong hàng nghìn tài liệu khớp khác.

Các bảng chi tiết cho từng phương pháp:

K	Precision@K	Recall@K	F1@K
1	0.4120	0.4120	0.4120
2	0.2685	0.5370	0.3580
3	0.2033	0.6100	0.3050
5	0.1358	0.6790	0.2263
10	0.0769	0.7695	0.1399

Bảng 3.3: Kết quả đánh giá mô hình BM25

K	Precision@K	Recall@K	F1@K
1	0.1305	0.1305	0.1305
2	0.0995	0.1990	0.1327
3	0.0778	0.2335	0.1167
5	0.0541	0.2705	0.0902
10	0.0306	0.3060	0.0556

Bảng 3.4: Kết quả đánh giá mô hình Boolean

K	Precision@K	Recall@K	F1@K
1	0.2195	0.2195	0.2195
2	0.1633	0.3265	0.2177
3	0.1323	0.3970	0.1985
5	0.0963	0.4815	0.1605
10	0.0627	0.6270	0.1140

Bảng 3.5: Kết quả đánh giá mô hình VSM

K	Precision@K	Recall@K	F1@K
1	0.4132	0.4132	0.4132
2	0.2716	0.5432	0.3621
3	0.2043	0.6128	0.3064
5	0.1349	0.6744	0.2248
10	0.0771	0.7708	0.1401

Bảng 3.6: Kết quả đánh giá ES với chỉ mục đã được xử lý

K	Precision@K	Recall@K	F1@K
1	0.4012	0.4012	0.4012
2	0.2644	0.5288	0.3525
3	0.1988	0.5964	0.2982
5	0.1339	0.6696	0.2232
10	0.0737	0.7372	0.1340

Bảng 3.7: Kết quả đánh giá ES với chỉ mục bình thường

3.5.2 Nhận xét kết quả

Từ kết quả đánh giá, có thể rút ra một số nhận xét quan trọng sau đây:

So sánh BM25 và ES đã xử lý: Hai phương pháp có hiệu suất gần như tương đương với MRR chênh lệch chỉ 0.0015. ES nhỉnh hơn một chút do thuật toán BM25 trong Elasticsearch đã được tối ưu hóa. Cả hai đều đạt xấp xỉ 77% Recall@10, nghĩa là trong top 10 kết quả có chứa khoảng 77% tài liệu liên quan - một kết quả rất tốt với tập dữ liệu có tỷ lệ positive thấp. Điều này chứng minh rằng thuật

toán BM25 cài đặt thủ công hoàn toàn có thể cạnh tranh được với các hệ thống tìm kiếm thương mại. Sự chênh lệch nhỏ giữa hai phương pháp có thể đến từ việc Elasticsearch sử dụng nhiều tối ưu hóa bổ sung như caching, bloom filters, và các thuật toán phân tán.

So sánh hai cách đánh chỉ mục của ES: ES với chỉ mục đã được xử lý tách từ cho kết quả tốt hơn. Cụ thể, MRR tăng từ 0.5136 lên 0.5284, tương đương với mức cải thiện 2.88%. Điều này chứng tỏ tiền xử lý văn bản tiếng Việt giúp cải thiện hiệu suất tìm kiếm đáng kể. Khi truy vấn được tách từ (tokenize) trước bằng underthesea, Elasticsearch sẽ tìm chính xác hơn các từ ghép như "truy_cứu", "trách_nhiệm", "hình_sự" thay vì tìm các token riêng lẻ không có nghĩa.

Sự khác biệt rõ rệt nhất ở Recall@10: ES đã xử lý đạt 77.08% trong khi ES bình thường chỉ đạt 73.72%. Điều này có nghĩa là với cùng 10 kết quả trả về, ES đã xử lý tìm được nhiều tài liệu liên quan hơn 3.36%. Với tập dữ liệu mà mỗi truy vấn chỉ có 1-3 tài liệu liên quan trong hàng trăm nghìn tài liệu, sự cải thiện này là đáng kể.

BM25 vs VSM: BM25 vượt trội hơn VSM một cách đáng kể với MRR 0.5269 so với 0.3353 (cải thiện 57.1%). Nguyên nhân chính đến từ các đặc tính tốt hơn của BM25:

- **Độ bão hòa TF (TF saturation):** Trong VSM, khi một từ xuất hiện nhiều lần trong tài liệu, điểm số tăng tuyến tính. Điều này có thể khiến các tài liệu dài với nhiều lần lặp lại của từ khóa bị đẩy lên cao dù không thực sự liên quan. BM25 giải quyết vấn đề này bằng hàm bão hòa, làm giảm ảnh hưởng của từ xuất hiện quá nhiều lần.
- **Tham số chuẩn hóa độ dài tài liệu (b):** BM25 sử dụng tham số b để cân bằng giữa văn bản dài và ngắn. VSM không có cơ chế này, dẫn đến các tài liệu dài có xu hướng được ưu tiên hơn do có nhiều từ khớp hơn.
- **IDF bão hòa:** BM25 sử dụng công thức IDF có tính bão hòa, trong khi VSM sử dụng IDF log đơn giản. Điều này giúp BM25 xử lý tốt hơn các từ rất phổ biến trong corpus.

Mô hình Boolean: Mô hình Boolean có hiệu suất thấp nhất trong các phương pháp được đánh giá. MRR chỉ đạt 0.1900, thấp hơn 3 lần so với BM25. Nguyên nhân chính là không có xếp hạng - tất cả tài liệu liên quan được coi là có cùng mức độ liên quan. Khi một truy vấn có nhiều từ (ví dụ: "phạm tội và trộm cắp và tài sản"), kết quả trả về có thể rất ít. Ngược lại, khi truy vấn ngắn (ví dụ: "luật"), kết quả trả về có thể rất nhiều (hàng nghìn tài liệu). Điều này dẫn đến việc tài liệu liên quan thực sự có thể bị chìm trong hàng nghìn tài liệu khác.

Precision@1 của Boolean chỉ đạt 13.05%, có nghĩa là chỉ khoảng 1/8 truy vấn có tài liệu liên quan ở vị trí đầu tiên. Đây là hệ quả trực tiếp của việc không có xếp hạng - tài liệu liên quan có thể nằm ở bất kỳ vị trí nào trong danh sách kết quả.

Xu hướng khi K thay đổi: Khi K tăng:

- Precision@K giảm (vì thêm nhiều tài liệu không liên quan).
- Recall@K tăng (vì tìm được nhiều tài liệu liên quan hơn).
- Đây là đặc tính cơ bản của Precision-Recall trade-off.

Cụ thể, với BM25:

- Precision@1 = 41.20% → Precision@10 = 7.69% (giảm 81.3%)
- Recall@1 = 41.20% → Recall@10 = 76.95% (tăng 86.8%)

Xu hướng này cho thấy nếu người dùng chỉ xem kết quả đầu tiên, cần hệ thống có Precision@1 cao. Nếu người dùng sẵn sàng xem nhiều kết quả, Recall@10 cao sẽ đảm bảo hầu hết tài liệu liên quan được hiển thị. Trong thực tế, người dùng thường xem 5-10 kết quả đầu tiên, do đó Recall@10 và MRR là các chỉ số quan trọng nhất.

Phân tích lỗi: Mặc dù kết quả tổng thể khả quan, vẫn còn một số trường hợp hệ thống hoạt động chưa tốt:

- **Truy vấn dài:** Khi truy vấn có nhiều từ, khả năng tìm được tài liệu khớp giảm do yêu cầu tất cả các từ đều phải xuất hiện.
- **Từ đồng nghĩa:** Hệ thống không xử lý từ đồng nghĩa, ví dụ "phạm tội" và "tội phạm" được coi là hai từ khác nhau.
- **Từ không có trong corpus:** Nếu truy vấn chứa từ không xuất hiện trong bất kỳ tài liệu nào, kết quả sẽ rất hạn chế.

3.6 Giao diện người dùng (UI)

Đề tài xây dựng giao diện người dùng để chạy thử nghiệm và trình bày kết quả. Giao diện được xây dựng dựa trên thư viện Streamlit [5], một thư viện Python cho phép tạo giao diện web nhanh chóng từ các script. Streamlit đặc biệt phù hợp cho các dự án nghiên cứu và thử nghiệm vì nó cho phép tạo giao diện tương tác mà không cần viết code frontend riêng biệt.

Lý do chọn Streamlit cho đề tài:

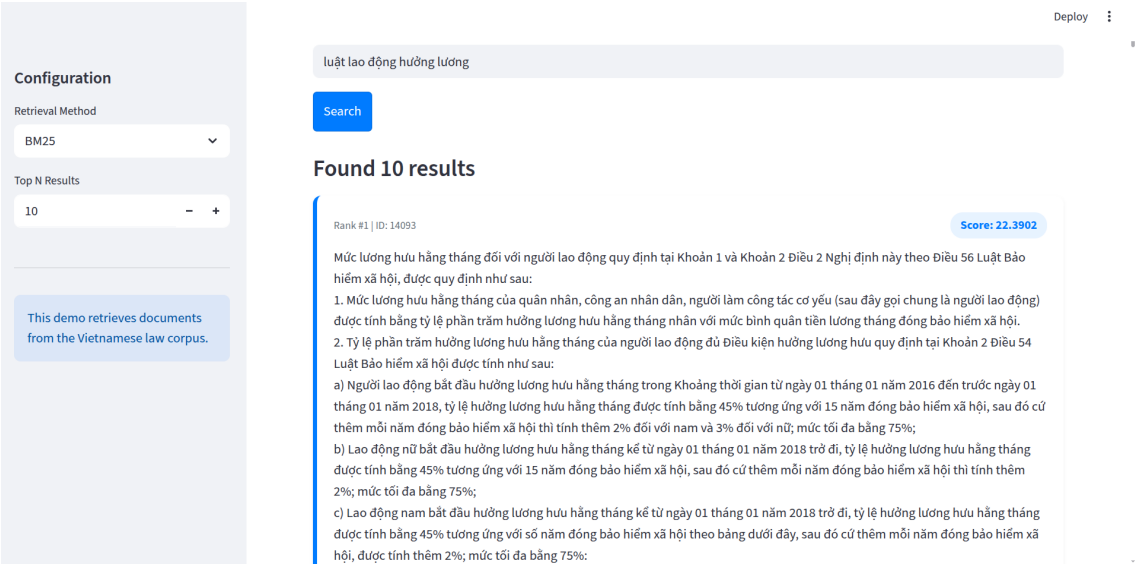
- Dễ dàng tích hợp với các model và xử lý dữ liệu Python.
- Không cần viết HTML/CSS/JavaScript.

- Hỗ trợ tự động tạo giao diện responsive.
- Có sẵn các component như sidebar, text input, select box, table, v.v.
- Cho phép hiển thị kết quả dưới dạng bảng, chart, văn bản.

Các tính năng của giao diện:

1. **Chọn phương pháp tìm kiếm:** Sidebar cho phép người dùng chọn 1 trong 5 phương pháp:
   - Boolean
   - Mô hình không gian vector (VSM)
   - BM25 (mặc định)
   - Elasticsearch (Thông thường)
   - Elasticsearch (Đã xử lý)
2. **Chọn số kết quả:** Ô chọn top N kết quả trả về (mặc định là 10).
3. **Nhập truy vấn:** Ô nhập câu hỏi tìm kiếm với placeholder gợi ý.
4. **Nút tìm kiếm:** Bắt đầu tìm kiếm khi người dùng nhập xong.
5. **Hiển thị kết quả:** Bao gồm:
   - Rank: Thứ tự (1, 2, ...)
   - Document ID: ID của văn bản trong corpus
   - Score: Điểm số (hoặc "N/A (Match)" cho Boolean)
   - Nội dung: Văn bản gốc đầy đủ

Giao diện chương trình minh họa như trên hình 3.1.



Hình 3.1: Giao diện chương trình minh họa kết quả

Quy trình xử lý khi người dùng thực hiện tìm kiếm:

1. Người dùng nhập câu hỏi vào ô tìm kiếm.
2. Nhấn nút "Tìm kiếm" hoặc nhấn Enter.
3. Hệ thống tiền xử lý câu hỏi (tách từ, chuẩn hóa).
4. Gọi phương pháp tìm kiếm đã chọn.
5. Nhận kết quả và hiển thị lên giao diện.

Một số lưu ý khi sử dụng giao diện:

- Với phương pháp Boolean, người dùng có thể sử dụng các toán tử AND, OR, NOT (viết hoa).
- Với các phương pháp khác, người dùng nhập câu hỏi bình thường.
- Thời gian tìm kiếm phụ thuộc vào phương pháp và độ phức tạp của truy vấn.
- Kết quả được sắp xếp theo điểm số (ngoại trừ Boolean).

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Tổng kết

Đề tài đã hoàn thành việc xây dựng hệ thống truy xuất thông tin văn bản luật Việt Nam với các thành phần chính:

1. **Hệ thống tiền xử lý văn bản tiếng Việt**: Sử dụng thư viện underthesea [6] để tách từ, chuẩn hóa văn bản và loại bỏ từ dừng.
2. **Chỉ mục ngược**: Xây dựng chỉ mục ngược phục vụ cho việc truy vấn nhanh chóng.
3. **Các mô hình truy xuất**:
   - Mô hình Boolean: Truy vấn sử dụng toán tử logic AND, OR, NOT.
   - Mô hình không gian vector (VSM): Sử dụng TF-IDF và cosine similarity.
   - Mô hình BM25: Sử dụng công thức BM25 với các tham số $k_1 = 1.5, b = 0.75$.
4. **Elasticsearch**: Xây dựng hệ thống tìm kiếm với 2 loại chỉ mục (đã xử lý và chưa xử lý).
5. **Giao diện người dùng**: Xây dựng giao diện web sử dụng Streamlit [5].

4.2 Kết quả đạt được

Kết quả đánh giá trên 2000 truy vấn cho thấy:

- ES (đã xử lý) đạt MRR = 0.5284, MAP = 0.5284.
- BM25 đạt MRR = 0.5269, MAP = 0.5269.
- Hai phương pháp này vượt trội hơn so với VSM (MRR = 0.3353) và Boolean (MRR = 0.1900).

Điều đáng chú ý là việc tiền xử lý văn bản tiếng Việt (tách từ bằng underthesea) giúp cải thiện đáng kể kết quả tìm kiếm của Elasticsearch.

4.3 Hạn chế

Một số hạn chế của đề tài:

- Chi phí tính toán cao với tập dữ liệu lớn (222981 tài liệu).
- Chưa xử lý các trường hợp từ đồng âm, từ đồng nghĩa.
- Mô hình Boolean chưa hỗ trợ nhiều toán tử phức tạp.
- Chưa áp dụng các kỹ thuật học sâu như BERT, biểu diễn ngữ nghĩa.

4.4 Hướng phát triển

1. Tích hợp các mô hình ngôn ngữ lớn (LLM) như BERT, PhoBERT để biểu diễn ngữ nghĩa tốt hơn.
2. Xây dựng hệ thống reranking để cải thiện thứ hạng kết quả.
3. Tăng cường xử lý từ đồng nghĩa, từ đồng âm trong tiếng Việt.
4. Tối ưu hóa hiệu suất cho tập dữ liệu lớn hơn.
5. Triển khai hệ thống với giao diện web thực tế.

TÀI LIỆU THAM KHẢO

[1] C. D. Manning, P. Raghavan, and H. Sch"utze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008, ISBN: 9780521865715. [Online]. Available: http://nlp.stanford.edu/IR-book/information-retrieval-book.html

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. DOI: 10.1561/1500000019

[3] E. B.V., *Elasticsearch: Theofficial distributed search & analytics engine*, https://www.elastic.co/elasticsearch/, 2024.

[4] YulTC, *Vietnamese legal documents*, https://huggingface.co/datasets/YuITC/Vietnamese-Legal-Documents/tree/main.

[5] S. Inc., *Streamlit: Turn data scripts into shareable web apps*, https://streamlit.io/, 2024.

[6] V. Anh, *Underthesea: Vietnamese nlp toolkit*, https://github.com/undertheseanlp/underthesea, 2020.